

Grundlagen der Algorithmen

Zusammenfassung und Lernzettel

Kompakte Zusammenfassung der Vorlesungsinhalte für die Prüfungsvorbereitung.

Autor Levin Rüßmann
Matrikelnummer 838791
Studiengang Informatik B.Sc.
Semester Sommersemester 2026
Datum 11.07.2026

Inhaltsverzeichnis

1 Informations- und Hinweis-Boxen	2
1.1 infobox — Infos und Definitionen	2
1.2 warnbox — Warnhinweise und Fallstricke	2
1.3 merksatz — Wichtige Sätze und Regeln	2
1.4 beispiel — Konkrete Beispiele	2
1.5 aufgabe — Übungsaufgaben	3
2 Strukturelemente	4
2.1 process — Schrittfolgen	4
2.2 procon — Vor- und Nachteile	4
2.3 konzeptkarte — Strukturierte Übersichtskarte	4
3 Code und Tabellen	5
3.1 Inline-Code und Codeblöcke	5
3.2 stdtable — Formatierte Tabellen	5
4 Grundlagen der Algorithmen	6
4.1 Definition	6
4.2 Eigenschaften eines korrekten Algorithmus	6
4.3 Vorteile und Nachteile formaler Algorithmen	6
4.4 Wichtiger Hinweis zur Komplexitätsanalyse	7
5 Laufzeitkomplexität	8
5.1 Überblick und Notation	8
5.2 Abbildung: Wachstum der O-Klassen	8
5.3 Vergleich bekannter Sortieralgorithmen	8
5.3.1 Gemessene Laufzeiten (Daten aus R)	9
5.3.2 Zwei Tabellen nebeneinander (zweispaltig)	9
6 Implementierungsbeispiel	10
6.1 Merge Sort in Python	10
6.2 Laufzeit-Analyse des Merge-Sort-Schritts	10

1 Informations- und Hinweis-Boxen

1.1 infobox — Infos und Definitionen

Definition: Algorithmus

Ein **Algorithmus** ist eine endliche, eindeutige Folge von Anweisungen, die ein Problem in endlicher Zeit löst. Jeder Algorithmus muss *terminieren*, *korrekt* und *eindeutig* sein.

Infoboxen können auch ohne Titel verwendet werden — z. B. für kurze Hinweise oder ergänzende Informationen im Fließtext.

1.2 warnbox — Warnhinweise und Fallstricke

Off-by-one-Fehler

Beim Zugriff auf Array-Indizes stets prüfen, ob der Index im Bereich $[0, n - 1]$ liegt. Ein häufiger Fehler ist der Zugriff auf Index n , was zu einer `IndexOutOfBoundsException` führt.

1.3 merksatz — Wichtige Sätze und Regeln

Master-Theorem

Für Rekurrenzen der Form $T(n) = a \cdot T(n/b) + f(n)$ gilt:

$$T(n) = \begin{cases} \Theta(n^{\log_b a}) & \text{falls } f(n) = O(n^{\log_b a - \varepsilon}) \\ \Theta(n^{\log_b a} \log n) & \text{falls } f(n) = \Theta(n^{\log_b a}) \\ \Theta(f(n)) & \text{falls } f(n) = \Omega(n^{\log_b a + \varepsilon}) \end{cases}$$

Merksätze können auch ohne Titel stehen — etwa für kurze, prägnante Regeln, die man unbedingt im Kopf behalten muss.

1.4 beispiel — Konkrete Beispiele

Beispiel: Quicksort auf [3, 1, 4, 1, 5, 9]

Pivot = 3 (erstes Element). Partition ergibt:

- Links ≤ 3 : [1, 1]
- Rechts > 3 : [4, 5, 9]

Rekursiver Aufruf auf beide Teile, dann zusammensetzen: [1, 1, 3, 4, 5, 9].

1.5 aufgabe – Übungsaufgaben

Aufgabe 1: Laufzeitanalyse

Bestimmen Sie die durchschnittliche Zeitkomplexität von Quicksort mithilfe des Master-Theorems. Welche Rekurrenz beschreibt den Algorithmus?

Aufgabe 2

Implementieren Sie Mergesort in Python und vergleichen Sie die Laufzeit empirisch mit Bubblesort für $n \in \{100, 1000, 10000\}$.

2 Strukturelemente

2.1 process – Schrittfolgen

- 1 Pivot-Element aus der Liste wählen (z. B. erstes, letztes oder mittleres)
- 2 Liste in zwei Teile partitionieren: Elemente $\leq$ Pivot links, $>$ Pivot rechts
- 3 Quicksort rekursiv auf den linken Teilbereich anwenden
- 4 Quicksort rekursiv auf den rechten Teilbereich anwenden
- 5 Ergebnis: links + [pivot] + rechts

2.2 procon – Vor- und Nachteile

Vorteile

- Sehr schnell in der Praxis ($O(n \log n)$ im Durchschnitt)
- In-place — kein zusätzlicher Speicher nötig
- Cache-freundlich durch sequentiellen Speicherzugriff

Nachteile

- Worst-case $O(n^2)$ bei schlechter Pivot-Wahl
- Nicht stabil — gleiche Elemente können die Reihenfolge tauschen
- Rekursionstiefe kann bei großen n zum Stack-Overflow führen

2.3 konzeptkarte – Strukturierte Übersichtskarte

Die **konzeptkarte** fasst Beschreibung, Tabelle und Ablauf in einer visuellen Karte zusammen. Mit **zweispaltig** lassen sich zwei Inhalte nebeneinander setzen:

Quicksort

Teile-und-herrsche-Verfahren: Die Liste wird an einem Pivot-Element partitioniert und beide Hälften werden rekursiv sortiert.

Eigenschaft	Wert
Stabil	Nein
In-place	Ja
Average	$O(n \log n)$

- 1 Pivot wählen
- 2 Partitionieren
- 3 Rekursiv sortieren

3 Code und Tabellen

3.1 Inline-Code und Codeblöcke

Für Inline-Code genügen Backticks: Die Funktion `quicksort(arr)` gibt ein sortiertes Array zurück. Typst bringt Syntax-Highlighting von Haus aus mit — einfach die Sprache hinter die öffnenden Backticks schreiben:

```
def quicksort(arr):  
    if len(arr) <= 1:  
        return arr  
    pivot = arr[0]  
    links = [x for x in arr[1:] if x <= pivot]  
    rechts = [x for x in arr[1:] if x > pivot]  
    return quicksort(links) + [pivot] + quicksort(rechts)
```

3.2 stdtable — Formatierte Tabellen

Algorithmus	Average Case	Notiz
Bubble Sort	$O(n^2)$	Nur für sehr kleine Eingaben
Merge Sort	$O(n \log n)$	Stabil, nicht in-place
Quick Sort	$O(n \log n)$	In der Praxis am schnellsten

4 Grundlagen der Algorithmen

4.1 Definition

Definition: Algorithmus

Ein **Algorithmus** ist eine endliche, eindeutige Folge von Anweisungen, die ein Problem in einer endlichen Anzahl von Schritten löst. Er besitzt stets eine klare Eingabe, eine definierte Ausgabe und terminiert nach endlich vielen Schritten.

Algorithmen sind das Herzstück der Informatik. Seit Dijkstra 1959 seinen wegweisenden Kürzeste-Wege-Algorithmus vorstellte, ist die formale Analyse von Algorithmen ein eigenständiges Forschungsfeld.

4.2 Eigenschaften eines korrekten Algorithmus

Ein Algorithmus muss folgende Eigenschaften erfüllen:

- 1 **Finitheit:** Der Algorithmus endet nach einer endlichen Anzahl von Schritten unter allen zulässigen Eingaben.
- 2 **Eindeutigkeit:** Jede Anweisung ist klar und unmissverständlich definiert; kein Schritt lässt Interpretationsspielraum.
- 3 **Ausführbarkeit:** Alle Schritte sind mit den zur Verfügung stehenden Mitteln tatsächlich durchführbar.
- 4 **Determiniertheit:** Gleiche Eingaben liefern stets dasselbe Ergebnis (bei deterministischen Algorithmen).
- 5 **Effizienz:** Der Ressourcenverbrauch (Zeit, Speicher) ist in einem akzeptablen Verhältnis zur Problemgröße.

4.3 Vorteile und Nachteile formaler Algorithmen

Vorteile

- Wiederverwendbar und übertragbar auf verschiedene Probleminstanzen
- Formale Korrektheitsnachweise möglich
- Laufzeit- und Speicherverbrauch systematisch analysierbar
- Unabhängig von Programmiersprache oder Plattform beschreibbar

Nachteile

- Formale Beschreibung kann komplex und schwer lesbar werden
- Optimierung für konkrete Hardware oft notwendig
- Nicht alle Probleme sind algorithmisch lösbar (Halteproblem)
- Erheblicher Aufwand für den Korrektheitsbeweis

4.4 Wichtiger Hinweis zur Komplexitätsanalyse

Achtung: Asymptotische Notation

Die **O-Notation** beschreibt das *asymptotische* Wachstum und ignoriert konstante Faktoren sowie untere Terme. Ein Algorithmus mit $O(n \log n)$ kann für kleine n langsamer sein als ein $O(n^2)$ -Algorithmus mit kleinen Konstanten! Stets auch den *praktischen* Anwendungsfall berücksichtigen.

5 Laufzeitkomplexität

5.1 Überblick und Notation

Wichtige O-Klassen (von schnell nach langsam)

$O(1)$	konstant
$O(\log n)$	logarithmisch
$O(n)$	linear
$O(n \log n)$	quasilinear
$O(n^2)$	quadratisch
$O(2^n)$	exponentiell
$O(n!)$	faktoriell

5.2 Abbildung: Wachstum der O-Klassen

Die folgende Grafik wurde mit R erzeugt (`r/komplexitaet.R`) und liegt als PNG in `assets/images/` — so funktioniert die R-Anbindung des Templates.

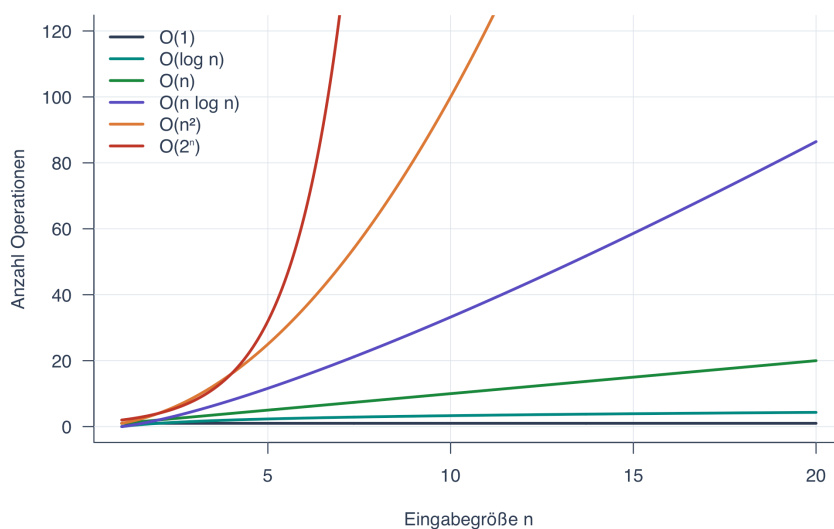


Abbildung 1. Wachstumsverhalten verschiedener Komplexitätsklassen für $n \in [1, 20]$. Quelle: eigene Darstellung, erzeugt mit R.

5.3 Vergleich bekannter Sortialgorithmen

Die folgende Tabelle vergleicht gebräuchliche Sortiervverfahren anhand ihrer asymptotischen Laufzeiten.

Algorithmus	Best Case	Average Case	Worst Case	Stabil?
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Ja
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	Nein
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	Ja
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Ja

Algorithmus	Best Case	Average Case	Worst Case	Stabil?
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	Nein
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	Nein
Counting Sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	Ja

5.3.1 Gemessene Laufzeiten (Daten aus R)

Diese Tabelle wird zur Kompilierzeit aus `assets/data/sortier_benchmark.csv` gelesen — die Datei stammt aus einem echten R-Benchmark (`make r`):

n	Bubble Sort (ms)	sort() in R (ms)
250	11.0	0.030
500	19.0	0.030
1000	81.0	0.045
2000	312.0	0.080

5.3.2 Zwei Tabellen nebeneinander (zweispaltig)

Interne Sortiervverfahren

Verfahren	In-Place?
Insertion Sort	Ja
Quick Sort	Ja
Merge Sort	Nein
Heap Sort	Ja

Externe Sortiervverfahren

Verfahren	Passes
Externes Merge	$O(\log n)$
Polyphasen-Merge	$O(\log n)$
Replacement Sel.	$O(n)$

6 Implementierungsbeispiel

6.1 Merge Sort in Python

Der folgende Code zeigt eine kompakte Python-Implementierung von Merge Sort.

```
def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(left, right):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    return result + left[i:] + right[j:]

# Beispielaufruf
data = [38, 27, 43, 3, 9, 82, 10]
print(merge_sort(data))  # -> [3, 9, 10, 27, 38, 43, 82]
```

6.2 Laufzeit-Analyse des Merge-Sort-Schritts

- 1 Teile das Array der Länge n in zwei Hälften der Länge $\lfloor n/2 \rfloor$ und $\lceil n/2 \rceil$.
- 2 Sortiere jede Hälfte rekursiv — Laufzeit je $T(n/2)$.
- 3 Füge die zwei sortierten Hälften zusammen — lineare Arbeit $O(n)$.
- 4 Die Rekurrenz $T(n) = 2T(n/2) + O(n)$ ergibt nach dem Master-Theorem $T(n) = O(n \log n)$.

Master-Theorem (Fall 2)

Gilt $T(n) = aT(n/b) + \Theta(n^{\log_b a})$, so ist $T(n) = \Theta(n^{\log_b a} \log n)$. Für Merge Sort: $a = 2, b = 2, f(n) = n = \Theta(n^{\log_2 2}) = \Theta(n)$, also $T(n) = \Theta(n \log n)$.